Project: NumPy Analyzer

Objective:

Develop a NumPy Analyzer that integrates Numpy functionalities and Object-Oriented Programming (OOP) principles. The toolkit will allow users to perform common data operations, statistical analyses, and mathematical computations on datasets using Numpy arrays.

Features & Requirements:

Class and Object Structure:

- Create a class DataAnalytics with methods to encapsulate all functionalities.

Array Management:

- Creation, Indexing, and Slicing: Allow users to create 1D, 2D, or 3D arrays and access specific elements, rows, columns, or slices of the array.
- Combining and Splitting: Provide functionality to concatenate multiple arrays or split an array into smaller arrays.

Mathematical Operations:

- Perform element-wise addition, subtraction, multiplication, and division on arrays.
- Calculate the dot product and matrix multiplication for 2D arrays.

Search, Sort, and Filter:

- Enable searching for specific values in an array.
- Sort arrays in ascending or descending order.
- Filter arrays based on user-defined conditions.

Aggregating Functions:

- Compute the sum, mean, median, standard deviation, and variance of the array elements.

Statistical Functions:

- Provide methods to calculate statistical properties like:
    - Minimum and maximum values.

- Percentiles.
- Correlation coefficients between arrays.

Object-Oriented Programming (OOP):

- Use constructors for initializing arrays.
- Include encapsulation and define private methods for internal computations.
- Implement class-level methods (@classmethod) and static methods (@staticmethod) for additional utility.

User Interface (UI):

- Create a menu-driven interface to let users choose from different options.
- Allow users to exit the program from the main menu.

Example Console Interaction:

Welcome Start:

```
Welcome to the NumPy Analyzer!
======================================
Choose an option:
1. Create a Numpy Array
2. Perform Mathematical Operations
3. Combine or Split Arrays
4. Search, Sort, or Filter Arrays
5. Compute Aggregates and Statistics
6. Exit
Enter your choice: 1
```

Array Creation:

```
Select the type of array to create:
1. 1D Array
2. 2D Array
3. 3D Array
Enter your choice: 2

Enter the number of rows: 2
Enter the number of columns: 3
Enter 6 elements for the array separated by space: 10 20 30 40 50 60

Array created successfully:
[[10 20 30]
 [40 50 60]]
```

```
Choose an operation:
1. Indexing
2. Slicing
3. Go Back
Enter your choice: 2

Enter the row range (start:end): 0:2
Enter the column range (start:end): 1:3

Sliced Array:
[[20 30]
 [50 60]]
```

Mathematical Operations:

```
Choose a mathematical operation:
1. Addition
2. Subtraction
3. Multiplication
4. Division
Enter your choice: 1

Enter the same-size array elements (6 elements separated by space): 5 5 5 5
5 5

Original Array:
[[10 20 30]
 [40 50 60]]

Second Array:
[[ 5  5  5]
 [ 5  5  5]]

Result of Addition:
[[15 25 35]
 [45 55 65]]
```

Combine or Split Arrays:

```
Choose an option:
1. Combine Arrays
```

```
2. Split Array
Enter your choice: 1

Enter the elements of another array to combine (6 elements separated by
space): 1 2 3 4 5 6

Original Array:
[[10 20 30]
 [40 50 60]]

Second Array:
[[1 2 3]
 [4 5 6]]

Combined Array (Vertical Stack):
[[10 20 30]
 [40 50 60]
 [ 1  2  3]
 [ 4  5  6]]
```

Search, Sort, and Filter:

```
Choose an option:
1. Search a value
2. Sort the array
3. Filter values
Enter your choice: 2

Original Array:
[[10 20 30]
 [40 50 60]]

Sorted Array:
[[10 20 30]
 [40 50 60]]
(Sorting applied row-wise.)
```

Aggregates and Statistics:

```
Choose an aggregate/statistical operation:
1. Sum
2. Mean
```

```
3. Median
4. Standard Deviation
5. Variance
Enter your choice: 3

Original Array:
[[10 20 30]
 [40 50 60]]

Median of Array: 35.0
```

Exit:

```
Choose an option:
1. Create a Numpy Array
2. Perform Mathematical Operations
3. Combine or Split Arrays
4. Search, Sort, or Filter Arrays
5. Compute Aggregates and Statistics
6. Exit
Enter your choice: 6

Thank you for using the NumPy Analyzer! Goodbye!
```

Instructions:

Task Completion: Ensure that you attempt all the assigned tasks given to you as part of the exam. Complete each task to the best of your ability, following the instructions provided.

Assumptions: Make suitable assumptions wherever necessary, based on the requirements and instructions provided. Document any assumptions made in your project documentation or README.md file.

GitHub Repository: Create a GitHub repository to host your project. Upload your project files, including source code, and documentation to the repository. Ensure that you provide a clear and descriptive README.md file.

No Copying: Do not copy code or any other content from your classmates or any other source. Plagiarism is strictly prohibited and can result in severe consequences, including academic penalties. Ensure that all the code and content in your project are original and properly attributed to the appropriate sources, if applicable.

Submission: Once you have completed your project and uploaded it to your GitHub repository, submit the GitHub repository link to your instructor or as instructed. Double-check that your repository is properly organized and includes all the required files as per the instructions provided.

Remember to follow the instructions provided professionally, make suitable assumptions wherever necessary, and avoid copying code or content from unauthorized sources. Good luck with your project work!